

A High-Throughput and Flexible CNN Accelerator Based on Mixed-Radix FFT Method

Yishuo Meng, Junfeng Wu, Siwei Xiang, Jianfei Wang^{1b}, Jia Hou, Zhijie Lin, and Chen Yang^{1b}, *Member, IEEE*

Abstract—CNN acceleration algorithms, including Winograd, Fast Fourier Transform (FFT) and Number Theoretic transform (NTT), have demonstrated their potential in efficiently operating current Convolutional Neural Networks (CNNs). However, deploying FFT algorithm for CNN acceleration would introduce significant invalid elements, unnecessary computations and unacceptable transformation overhead. To address these issues, this paper proposes a series of improved methods along with an FFT-based architecture for efficient and simplified CNN acceleration. First, a novel mixed-radix FFT algorithm is proposed for the reduction of invalid elements. Moreover, Hermitian symmetry is utilized to further reduce the scale of FFT transformation and the number of multiplications. Furthermore, an efficient FFT-based CNN accelerator with a resource-efficient transformation component and a multiplication-reduced PE array is designed. Our proposed accelerator is implemented based on Xilinx XCVU440 with a running frequency of 238MHz, achieving actual performance of 2109-2797 GOPS and DSP efficiency of 1.37-1.82 GOPS/DSP. Compared to previous works based on Winograd, FFT and NTT, our proposed accelerator can realize up to 9.42 $\times$ speedup on actual performance and 1.11 $\times$ – 6.41 $\times$ speedup on DSP efficiency.

Index Terms—Convolutional neural network, fast Fourier transform, Hermitian symmetry, DSP efficiency.

I. INTRODUCTION

STATE-OF-THE-ART CNNs have demonstrated superior performance across diverse domains, including image classification [1], [2], [3], autonomous driving [4], [5] and object recognition [6]. However, the increasing demands for storage and computation of current CNNs pose challenges in their deployment within resource-constrained scenarios. Consequently, convolution acceleration algorithms, including Winograd [7], [8], [9], FFT [10], [11] and NTT [12], are commonly employed to optimize the inference of CNNs while customizing hardware accelerators. Among these algorithms, Winograd algorithm is always selected, applied and extended to further enhance the performance of CNN accelerators [8], [9], [13]. Considering the factors such as multiplication savings and the range of transformation constants, the applicability of the Winograd algorithm for current CNN acceleration

Received 1 May 2024; revised 28 July 2024; accepted 14 September 2024. Date of publication 4 October 2024; date of current version 30 January 2025. This work was supported by the National Natural Science Foundation of China under Grant 62176206. This article was recommended by Associate Editor X. Fong. (Corresponding author: Chen Yang.)

The authors are with the School of Microelectronics, Xi'an Jiaotong University, Xi'an, Shaanxi 710049, China (e-mail: 3121156027@stu.xjtu.edu.cn; wjf2016@stu.xjtu.edu.cn; X1004312107@stu.xjtu.edu.cn; jfwang@stu.xjtu.edu.cn; hjwst0314@stu.xjtu.edu.cn; linzhijie@stu.xjtu.edu.cn; chyang00@xjtu.edu.cn).

Digital Object Identifier 10.1109/TCSI.2024.3466563

Winograd algorithm						
n	r	m	max	min	Bit number	Multiplication saving
4	3	2	1	1/2	2	2.25
5	3	3	4	1/6	10	3.24
6	3	4	8	1/24	11	4
7	3	5	81	1/120	18	4.59
8	3	6	243	1/720	21	5.06
9	3	7	4096	1/5040	29	5.44
6	5	2	5	1/24	11	2.78
7	5	3	16	1/120	16	4.59
8	5	4	49	1/720	19	6.25
9	5	5	256	1/5040	25	7.71
FFT algorithm						
4	3	2	1	0.25	3	1.64
8	3	6	1	0.088	11	3.45
16	3	14	1	0.024	11	4.62
32	3	30	1	0.0061	13	5.28

n: Input size r: Filter size m: Output size max/min: the maximum/minimum constant

Fig. 1. The comparisons of the multiplication saving and precision requirement for Winograd and FFT algorithm.

may be limited. Specifically, as illustrated in Fig. 1, previous work [11] have observed that compared to FFT algorithm with the same input size, the Winograd algorithm imposes higher precision requirements. Furthermore, our comprehensive comparison also reveals that while achieving similar multiplication saving (e.g., 4.59 vs. 4.62, 5.06 vs. 5.28), the transformation constants of the Winograd algorithm still necessitate more stringent precision requirements and a greater number of representation bits (e.g., 18 vs. 11 and 21 vs. 13) than that of FFT algorithm. To solve this problem, while operating the convolutions with large filters (e.g., 5×5 filters) or input tiles (e.g., 8×8 input tiles), current works [8], [9], [13], [38], [39] typically decompose them to smaller filters (e.g., 3×3 filters) with certain input tiles (e.g., 4×4 input tiles) instead of directly using the transformation constants that require more representation bits.

However, despite the lower precision requirements for the transformation constants [11], deploying FFT algorithm for CNN acceleration still presents several challenges. Firstly, there is a significant presence of invalid zero values introduced to FFT transformation from the expansion requirement for input feature maps (IFMs) and the alignment requirement for filters. Regarding the IFMs, previous accelerators have consistently employed the radix-2 FFT algorithm [7], [10], [11], necessitating that the transformation size be a power of two (2^n). Therefore, if IFM size is not 2^n (e.g., 14×14 or 28×28), zero expansion is applied, resulting in the introduction of invalid elements ($\sim 30.6\%$ invalid elements).

Additionally, the application of FFT algorithm also introduces superfluous calculations. These unnecessary calculations arise from the aforementioned invalid zero values and the complex multiplications required by the FFT algorithm. Consequently, the computation savings achieved by FFT algorithm are counteracted by the above unnecessary computations to some extent. Finally, apart from the unnecessary computations, the expansion of IFMs and alignment of filters would also introduce additional transformation overhead to the hardware implementation. Furthermore, it is found from Fig. 1 that in order to achieve the same multiplication savings, FFT algorithm requires a larger transformation scale. Therefore, the increasing transformation scale caused by the above two factors boosts the hardware overhead of FFT transformation and makes it difficult to deploy FFT algorithm for CNN acceleration.

In this work, first, an improved mixed-radix FFT algorithm is applied for the elimination of the invalid zero values and reduction of the transformation overhead. After reducing the invalid elements, Hermitian symmetry is employed to further reduce the FFT transformation scale. Through reducing the invalid elements and decreasing the transformation scale, both of the unnecessary computations and the hardware transformation overhead are reduced. Subsequently, a high-throughput CNN acceleration architecture with a resource-reduced transformation module and a multiplication-saved computation module is implemented for efficient and high-performance acceleration. With the aid of the booth multiplier and complex multiplication reduce scheme, the required multiplications as well as hardware overhead are greatly saved. The main contributions of this work are as follow:

- In this paper, after concluding the widely-used IFM sizes of current CNNs and selecting the suitable radixes, a mixed-radix FFT algorithm, including radix-2/3/4/5, is proposed to accomplish reconfigurable FFT transformation for a wide-range IFM sizes, thereby flexibly avoiding the IFM expansion and effectively reducing the invalid zero elements. The evaluation results show that by using the proposed mix-radix FFT algorithm, up to 68.4% invalid elements can be saved in IFM expansion.
- By leveraging the inherent property of real-valued operands in CNNs, Hermitian symmetry is applied to further reduce both of the transformation scale and required multiplications. Experimental results demonstrate that by harnessing Hermitian symmetry, the transformation scale and the number of multiplications can be reduced to up to 50% compared to the original FFT-based approaches.
- With the help of the above methods, a high-throughput CNN architecture with a resource-reduction transformation component and a multiplication-saving computation component is implemented. Specifically, booth multiplier is employed in the transformation module to provide hardware-level resource optimization. Moreover, a complex multiplication reduce scheme is also applied in the designed PE array for hardware-level computation reduction. Our proposed accelerator is implemented based on Xilinx XCVU440 platform with a running frequency of 238MHz, achieving 2109-2797 GOPS performance and 1.37-1.82 GOPS/DSP efficiency.

Different from the previous works that directly apply radix-2 FFT algorithm to CNN acceleration [7], [10], [11], we found that the existed IFM sizes are always not power-of-two (2^n), therefore, the conventional radix-2 FFT would introduce unneglectable invalid zero elements. To solve this problem, we analyze the generally-used IFM sizes and propose a mixed-radix FFT algorithm including radix-2/3/4/5 to flexibly support the various IFM sizes in CNNs and reduce the invalid zero elements. Consequently, the difference on radix would result in completely distinctions on algorithm design and hardware implementation.

In the meanwhile, it should be noted that radix-3 and radix-5 would consume more hardware resource (i.e., multipliers). Therefore, we firstly reduce the transformation scale to half of the origin by exploiting Hermitian symmetry to greatly save the hardware overhead. Furthermore, Booth multiplier is also applied in the implementation of transformation component to replace the DSPs with logic resources, achieving the further hardware overhead saving.

II. RELATED WORKS AND MOTIVATION

In this section, recent CNN accelerators that employ convolution acceleration algorithms including Winograd [7], [8], [9], [11], [13], [14], [15], [16], [17], [18], [19], [20], [21], [22] and FFT [10], [11], [23], [24], [25], [26], [27], [28] are discussed. Subsequently, the motivations of the proposed mixed-radix FFT algorithm and high-throughput CNN acceleration architecture are elaborated in detail.

A. The Acceleration of CNNs Based on Winograd

Winograd [15] proposed an improvement algorithm for convolutions, known as Winograd algorithm, which was later applied to the acceleration of CNNs in current works [7], [8], [9], [16], [17], [18], [19], [20], [21], [22]. In order to reduce computations and achieve high-performance acceleration, some specific and efficient Winograd units were employed [11]. Moreover, Winograd was also used to implement a CNN accelerator for video processing [16]. However, due to the evolving characteristic of CNNs, the application of Winograd algorithm to various convolution types (e.g., different filter sizes and strides) becomes challenging. Consequently, a filter decomposition method was proposed to unify the filter size and stride, achieving outperformed performance and simplified hardware structure while using a unified Winograd unit [8], [9]. Furthermore, as CNN pruning gradually emerges as a promising technique for efficient CNN acceleration, the combination of Winograd and sparse CNNs has been employed and sparse-Winograd CNNs have been proposed [17], [18]. Subsequently, the above sparse-Winograd CNNs were applied to CNN accelerator implementation to leverage the advantages of both CNN pruning and Winograd algorithm [19], [20], [21], [22].

B. The Acceleration of CNNs Based on FFT

FFT has traditionally been employed in digital signal processing and telecommunications [23], [24]. In recent years, there has been a growing trend for utilizing FFT for accelerating computation-intensive CNNs to reduce computation load [10], [11], [25], [26], [27], [28]. Specifically, previous

TABLE I
THE NON- 2^n IFM SIZE AND CORRESPONDING COMPUTATION
RATIO IN CURRENT CNNs

CNN	IFM size	Computation ratio
YOLOV1	28×28 14×14 7×7	63.31%
YOLOV3-tiny	26×26 13×13	75.82%
MobileNetV1	28×28 14×14 7×7	78.10%
ResNet-50	28×28 14×14 7×7	78.94%
GoogLeNetV1	28×28 14×14 7×7	69.8%

works, such as works [10] and [26] have utilized FFT for implementing the computation core and embedding it into the CNN acceleration system. Additionally, FFT processing element (PE) engine was designed for CNN acceleration in work [11]. Incorporating the advancing techniques for CNNs, such as 3D CNNs and CNN pruning, FFT was synergistically combined with the above techniques for the further performance enhancement. For instance, a novel FFT-based algorithm named F3D was proposed to significantly reduce computational complexity for 3D CNNs. Work [28] demonstrated that the key concept for those algorithms (i.e., Winograd and FFT) lies in reducing the multiplications at the cost of more additions, therefore an FFT-pruning method was employed to achieve a reduced-complexity fast FFT convolution algorithm.

C. Motivation of Our Work

Currently, deploying FFT algorithm to CNN acceleration still presents the following two main challenges.

First, there are still lots of invalid elements are induced in the commonly used radix-2 FFT transformation process due to the mismatch between the fixed transformation scale provided by radix-2 FFT algorithm and the flexible and various IFM sizes of contemporary CNNs. The radix-2 FFT algorithm has consistently been employed in prior FFT-based accelerators, realizing superior performance [7], [10], [11]. However, radix-2 FFT requires that the transformation scale should be 2^n , which implies that the IFM should be expanded to the latest $2^n \times 2^n$, resulting in the presence of significant invalid elements. Unfortunately, there are substantial number of IFMs whose scale is not 2^n , and these are according denominated as non-power-of-two IFMs (i.e., NPT IFMs). Specifically, Table I exhibits several examples of NPT IFMs in classical CNNs and their corresponding computation ratios. Taking YOLOv3-tiny as an example, the NPT IFMs whose scale is 13×13 or 26×26 would occupy 75.82% computations of the entire network. It is found that the backbone layers of these CNNs always need to be expanded to fulfill the necessities of FFT. Additionally, as illustrated in Table II, the expansion of NPT IFMs generates appreciable invalid elements, reaching up to 68.4%, resulting in unnecessary computations and increased transformation scale.

Moreover, the extra computations and hardware overhead induced by FFT also hinder its application on CNN acceleration. On the one hand, the aforementioned invalid zero elements caused by feature map expansion and filter alignment would easily introduce equip-proportional unnecessary computations. On the other hand, the utilization of FFT transformation results in a conversion of the initial real

TABLE II
THE INVALID ELEMENT RATIO WHILE USING
CONVENTIONAL RADIX-2 FFT

IFM size	Filter size/Stride	Post-padding IFM size	Transformation scale	Invalid element ratio
28×28	3×3/1	30×30	32×32	12.1%
28×28	5×5/1	32×32	32×32	0.0%
26×26	3×3/1	28×28	32×32	23.4%
14×14	3×3/1	16×16	16×16	0.0%
14×14	5×5/1	18×18	32×32	68.4%
13×13	3×3/1	15×15	16×16	12.1%
7×7	3×3/1	9×9	16×16	68.4%
7×7	5×5/1	11×11	16×16	52.7%

numbers into complex numbers. Thereby the corresponding complex multiplications would introduce $3 \times$ extra real multiplications [7], [11]. These extra computations caused by invalid zero elements and complex multiplications would make the benefits of FFT to be greatly reduced. Subsequently, the amplified scale of transformation exacerbates the issue of hardware overhead in FFT-based architecture, which cannot be disregarded. Apart from the increased transformation scale caused by IFM expansion and filter alignment, FFT algorithm needs to adopt a larger transformation scale to reflect promising multiplication reduction. Work [11] has observed that FFT has lower transformation overhead while comparing with Winograd. However, this comparison is based on the same transformation scale. Actually, for achieving similar multiplication saving, larger transformation scale should be applied in FFT-based accelerator. For example, $F(2 \times 2, 3 \times 3)$ is the most commonly-used transformation unit of Winograd [8], [9], and [14], while the frequently-used transformation scales for FFT are 4×4 or 8×8 [11]. Therefore, due to the increased transformation scale, deploying FFT into hardware implementation without any resource reduction method becomes challenging.

Therefore, first, an improved mixed-radix FFT algorithm is proposed to implement flexible transformation scales for IFMs. The available transformation scales are so close to the various IFM sizes that there is little need for feature map expansion. Through the aforementioned method, the invalid elements induced by conventional FFT are eliminated, realizing the reduction of corresponding multiplications and transformation overhead. After eliminating the invalid elements, Hermitian symmetry is applied to further reduce the transformation scale and save multiplications. Consequently, both of the unnecessary computations and transformation overhead are greatly reduced. Subsequently, an FFT-based CNN acceleration architecture with a resource-reduced transformation component and a multiplication-saved PE array is implemented for flexible and high-performance CNN acceleration.

III. ALGORITHM ANALYSIS AND IMPROVEMENT

In this section, first, an improved mixed-radix FFT algorithm is proposed to minimize the expansion requirement for IFMs. The aforementioned mixed-radix FFT method significantly reduces the occurrence of invalid elements caused by conventional radix-2 FFT algorithm, thereby reducing unnecessary multiplications and transformation overhead. Furthermore, Hermitian symmetry is employed after reducing the invalid elements to further optimize the transformation process

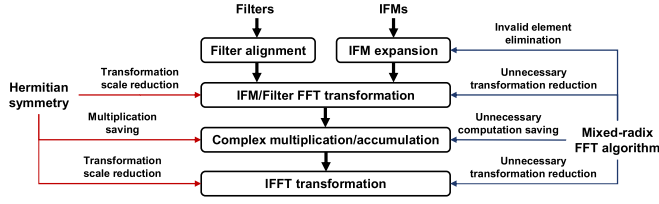


Fig. 2. The workflow using mixed-radix FFT algorithm for CNN acceleration.

and decrease the transformation scale. The above methods are finally integrated to evaluate the earnings from reducing invalid elements and decreasing the scale of transformation.

A. Proposed Mixed-Radix FFT Workflow and Radix Selection

Fig. 2 presents the total workflow of this work. Considering that the conventional radix-2 FFT would induce unacceptable invalid zero elements to IFM expansion, a mixed-radix FFT algorithm is proposed in this work. As indicated in TABLE II, the majority of the post-padding IFMs sizes in current CNNs range from 9×9 to 32×32 , with many of them being NPT. Consequently, expanding for these IFMs would result in a large number of invalid zero elements while using previous radix-2 FFT algorithm. The mixed-radix FFT algorithm, which has been successfully used in various fields such as 4G and WLAN [29], [30], [31], [32], is explored in this study for accelerating CNNs.

It is crucial to select appropriate radices to support most of the IFM sizes and minimize invalid zero expanding. First, we conclude the common-used NPT (i.e., non-power-of-two) IFMs and corresponding filter sizes in classical CNNs as the benchmarks of the radix selection. In addition, the padding of these IFMs is also considered. Subsequently, each of the post-padding IFMs is decomposed to multiple factors and the frequently-used factors are extracted as the radices. Therefore, as illustrated in Fig. 3, we try to decompose the post-padding IFM sizes in Table II and evaluate multiple radices, and radix-4 is also applied to shorten the transformation chain. Therefore, a mixed-radix FFT algorithm comprising of radix-2, radix-3, radix-4 and radix-5 is selected. However, two post-padding IFM sizes, including 28×28 and 11×11 cannot directly be supported by the chosen radices. Fortunately, they only need to be expanded to 30×30 and 12×12 instead of 32×32 and 16×16 , respectively.

B. Hermitian Symmetry-Based Transformation Simplification and Multiplication Reduction

Considering that all of the elements in the IFMs and filters are real numbers, their transformation results follow Hermitian symmetry. For 1D FFT transformation, assuming that the input sequence consists of N real number elements $x(0), x(1), \dots, x(n-1)$, the corresponding FFT transformation results are $X(0), X(1), \dots, X(N-1)$, and they can be depicted by the equation (3.1). Specifically, $X(1)$ is symmetric with $X(N-1)$, $X(2)$ is symmetric with $X(N-2)$. Thus only half of the transformation results need to be computed, while the remaining half can be directly generated through the conjugation. Moreover, in terms of 2D FFT transformation, the results can be described by equation (3.2). In this work, it is found that in (3.2), k_1 and k_2 cannot be zero, which means the first row and first column of the transformation

results are not included in equation (3.2). However, they also adhere 1D Hermitian symmetry and can be expressed using equation (3.3).

$$X(k) = X^*(N-k) \quad k \in (0, N-1] \quad (3.1)$$

$$X(k_1, k_2) = X^*(N-k_1, N-k_2) \quad k_1, k_2 \in (0, N-1] \quad (3.2)$$

$$\begin{cases} X(0, k_2) = X^*(0, N-k_2) & k_1 = 0 \text{ and } k_2 \neq 0 \\ X(k_1, 0) = X^*(N-k_1, 0) & k_2 = 0 \text{ and } k_1 \neq 0 \end{cases} \quad (3.3)$$

Consequently, as presented in Fig. 4(a) and Fig. 4(b), the grey elements of the transformation results are required to compute, while the white ones can be directly obtained. Specifically, as presented in Fig. 4(a), excluding the first row and first column, only the left two columns of the remaining 4×4 matrix needs to be computed while applying Hermitian symmetry. As for the circumstance of Fig. 4(b), there is a 5×5 matrix excluding the first row and first column, so only the two left columns and the first three elements of the third column need to be calculated. Furthermore, 2D Hermitian symmetry can also be employed for the multiplication reduction, this is because the FFT transformation results of both filters and IFMs follow Hermitian symmetry, according to equation (3.4), in this equation, “*” represents conjugate. Consequently, half of the multiplications of the transformed filters and IFMs can also be directly obtained by taking conjugate values of the multiplication results of the symmetric points. Hermitian symmetry has been applied for CNN acceleration and reduces the number of complex multiplications from $n \times n$ to $n \times ([n/2] + 1)$ in work [11], as shown in Fig. 4(c) and Fig. 4(d). However, this work only exploits the 2D Hermitian symmetry, thus failed to accomplish further multiplication reduction. In this work, the first row and first column of the transformation results which not follow 2D Hermitian symmetry are also exploited, thus achieve better multiplication reduction.

$$a \times b = (a^* \times b^*)^* \quad (3.4)$$

C. Proposed Mixed-Radix FFT Algorithm for CNN

In the above, there are totally two schemes introduced for the FFT transformation simplification and multiplication reduction. Specifically, mixed-radix FFT algorithm can efficiently decrease the IFM expansion and filter alignment, thereby reducing the invalid zero elements. Additionally, by leveraging Hermitian symmetry, half of the transformation and multiplication overhead is further saved, which is presented in Fig. 4. Moreover, the pseudo-code of the application of mixed-radix FFT algorithm to CNN acceleration is presented in Algorithm 1. Therefore, in this section, the aforementioned methods are evaluated jointly to show their effect on transformation overhead and multiplication requirement.

In terms of IFM transformation overhead, Table III presents the comparison between original radix-2 approach and proposed method using common convolution parameters. The results of this comparison demonstrate that by employing mixed-radix FFT algorithm and leveraging Hermitian symmetry, our approach can realize more than 50% reduction in IFM transformation overhead.

Finally, as presented in Table IV, in terms of the number of multiplications, mixed-radix FFT algorithm effectively reduces superfluous invalid zero elements and corresponding

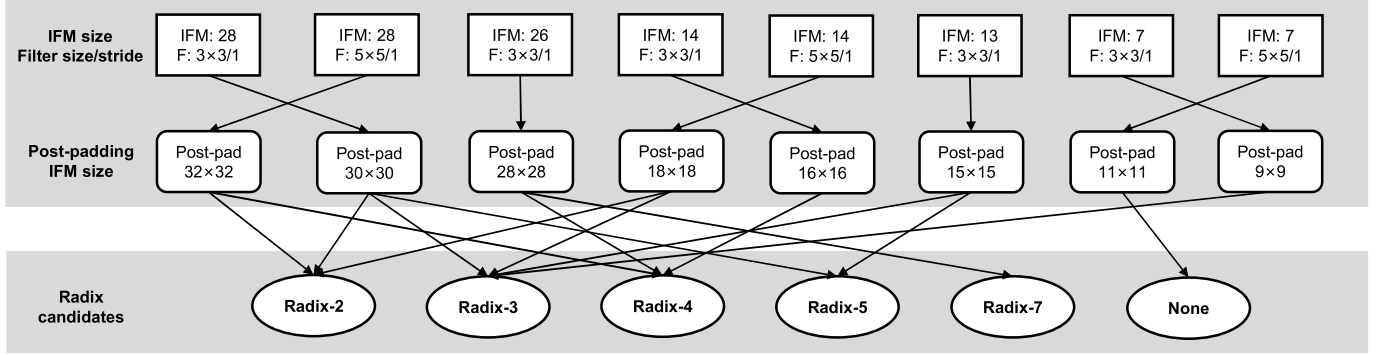


Fig. 3. The radix selection according to the post-padding IFM sizes.

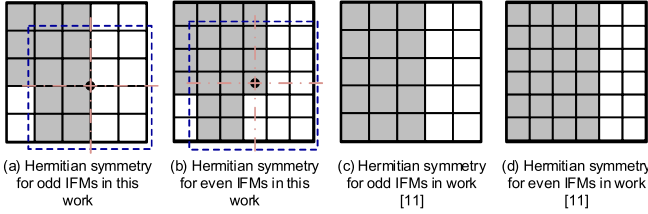


Fig. 4. The application of Hermitian symmetry.

Algorithm 1 Mixed-Radix FFT-Based CNN Acceleration

Input: IFMs; Filters;

Output: Output feature maps

/* Initialization */

```

1: for (row = 0; row < H; row += (t_size - k_size_row + 1) * tile_x)
2:   for (col = 0; col < W; col += (t_size - k_size_col + 1) * tile_y)
3:     for (t_o = 0; t_o < N; t_o += 8 * t_o_parallel)
4:       for (t_i = 0; t_i < M; t_i += t_i_parallel)
5:         for (t_o2 = 0; t_o2 < t_o2 + 8 * t_o_parallel; t_o2 += t_o_parallel)
6:           {tile_parallel = tile_x * tile_y;
7:             for (tile_N = 0; tile_N < tile_parallel; tile_N += 1)
8:               for (t_oN = 0; t_oN < t_o_parallel; t_oN += 1)
9:                 for (t_iN = 0; t_iN < t_i_parallel; t_iN += 1)
10:                  {load: tile_Z(row, col, tileN, t_i);
11:                   load: filter[t_i][t_o];
12:                   data = mr_FFT(tile_Z(row, col, tileN, t_i));
13:                   kernel = mr_FFT(filter[t_i][t_o]);
14:                   out(row, col, tileN, t_o) += (data * kernel);}
15:                 output(row, col, tileN, t_o) = mr_IFFT(out(row, col, tileN, t_o));}

```

TABLE III

THE IFM TRANSFORMATION OVERHEAD COMPARISON BETWEEN RADIX-2 FFT ALGORITHM AND THIS WORK

Parameter			Radix-2 FFT transformation		This work	
IFM size	Filter/Stride	Post-padding IFM size	Row-direct	Column-direct	Row-direct	Column-direct
28×28	3×3/1	30×30	32×32	32×32	30×16	16×30
28×28	5×5/1	32×32	32×32	32×32	32×17	17×32
26×26	3×3/1	28×28	32×32	32×32	30×16	16×30
14×14	3×3/1	16×16	16×16	16×16	16×9	9×16
14×14	5×5/1	18×18	32×32	32×32	18×10	10×18
13×13	3×3/1	15×15	16×16	16×16	15×8	8×15
7×7	3×3/1	9×9	16×16	16×16	9×5	5×9
7×7	5×5/1	11×11	16×16	16×16	12×7	7×12

unnecessary multiplications. Moreover, Hermitian symmetry can further achieve 50% computations reduction on the basis of mixed-radix FFT algorithm. Consequently, this study yields

computation savings ranging from 49.2%-84.0% compared with radix-2 FFT method.

Furthermore, we try to discuss how the above computation saving impact the hardware performance. Equation (3.5), as shown at the bottom of the next page, presents the computation formula of the hardware performance. It can be found that the actual performance includes four parts: the number of DSPs, the runtime frequency, the DSP utilization and the ratio of the theoretical computations to the actual computations. Therefore, it can be seen from the above formula that the computation saving achieved by algorithms can directly affect the improvement of the hardware performance. Consequently, this study yields computation savings ranging from 49.2%-84.0% compared with radix-2 FFT method, which means that the computation of our proposed algorithm is 16.0% (1-84.0%)-50.8% (1-49.2%) of the classical FFT algorithm. Consequently, our hardware can achieve $1.97 \times$ (i.e., $1/(1-49.2\%)$) to $6.25 \times$ (i.e., $1/(1-84.0\%)$) enhancement on hardware performance compared to the hardware based on classic radix-2 FFT algorithm under the condition of realizing the similar DSP number, runtime frequency and DSP utilization.

IV. HARDWARE IMPLEMENTATION

A. Overview

In this section, we present a detailed introduction to the applied overall computation dataflow and designed hardware implementation. The computation dataflow is illustrated in Fig. 5. First, after zero-padding, the IFMs are stored in on-chip buffer and read tile-by-tile using the storage component presented in Fig. 6 (i.e., the purple shade part), which includes DDR, on-chip IFM buffer and filter buffer. It should be noted that offline FFT transformation is performed for filters; hence, the transformed filters are directly stored in the on-chip filter buffer. Subsequently, the input tiles are sent to the transformation component (i.e., the green part in Fig. 6 to execute FFT transformation. Simultaneously, the transformed filters are also retrieved from the filter buffer. Next, the transformed input tiles and filters undergo element-wise multiplication and channel-wise accumulation, facilitated by the computation component (depicted in blue in Fig. 6). The outputs of the computation component are then subjected to IFFT transformation using the aforementioned transformation component. Following the transformation operation, post-processing tasks such as dequantization, activation and pooling are performed by the post-processing component (depicted in black in Fig. 6) to

TABLE IV
THE MULTIPLICATION COMPARISON BETWEEN GENERAL CONVOLUTION, RADIX-2 FFT ALGORITHM AND THIS WORK

IFM size	Filter/Stride	Post-padding IFM size	Multiplications (General)	Multiplications (Winograd)	Multiplications (Radix-2 FFT)	Multiplications (This work)	Saving ratio		
							General	Winograd	Radix-2 FFT
28×28	3×3/1	30×30	7056	3136	4096	1808	74.4%	42.3%	55.9%
28×28	5×5/1	32×32	19600	12544	4096	2056	89.5%	83.6%	49.8%
26×26	3×3/1	28×28	6084	3136	4096	1808	70.3%	42.3%	55.9%
14×14	3×3/1	16×16	1764	784	1024	520	70.5%	33.7%	49.2%
14×14	5×5/1	18×18	4900	3136	4096	656	86.6%	79.1%	84.0%
13×13	3×3/1	15×15	1521	784	1024	339	77.7%	56.8%	66.9%
7×7	3×3/1	9×9	441	256	1024	164	62.8%	35.9%	84.0%
7×7	5×5/1	11×11	1225	1024	1024	296	75.8%	71.1%	71.1%

generate the complete convolution or pooling results. Finally, the last write component (the red shade part in Fig. 6) is utilized to write the results back into the on-chip IFM buffer. In addition to the components associated with the computation dataflow, our implemented architecture also integrates a global controller and RAM controller. Specifically, the global controller would receive the CNN configuration encompassing all of the critical parameters of current CNN layers and distribute them to the above critical modules. Similar with the global controller, the CNN configuration is also exploited by RAM controller to schedule the input tile and filter access.

Moreover, fixed 9 precision is applied in the designed accelerator, and the main reasons of selecting fixed 9 precision are as follows:

First, as for the computation error, fixed 9 precision can accomplish negligible quantization and transformation error. For the evaluation of the computation error, Frobenius norm is applied in this study. Assuming that the input is a two-dimensional matrix A , the expression of Frobenius norm is shown in equation (4.1). Equation (4.2) defines the quantization error (QE), which represents the error brought by fixed-point quantization. In addition, this paper also defines the transformation error (TE), which is depicted by equation (4.3), to represent the discrepancy between results of the hardware and the $result_{standard}$. Furthermore, the evaluation method of work [28] is applied in this study, we generated 1000 groups of random vectors, and conducted error analysis on the FFT transformation. It can be concluded that both of the quantization error and conversion error are less than 0.5%, so it is proved that the error caused by 9-bit quantization is acceptable.

Second, as for the storage utilization, fixed 9 precision can fully exploit the capacity of Block RAM (BRAM) resource of Xilinx FPGA. It is found that the BRAM in XCVU440 FPGA is 36K. If Fixed 8 or Fixed 16 precision is applied, only 32K storage resource in a 36K BRAM is used, causing 11% BRAM wasting. Subsequently, when choosing fixed 9 precision, all of the 36K can be exploited. Therefore, fixed 9 precision not only can accomplish negligible quantization and transformation

error, but also can fully exploit the BRAM resource of Xilinx FPGA.

$$Frobenius(A) = \sqrt{\left(\sum_{i=1}^m \sum_{j=1}^n |a_{ij}|^2\right)} \quad (4.1)$$

$$QE = \frac{Frobenius(result_{standard} - result_{quant})}{Frobenius(result_{standard})} \quad (4.2)$$

$$TE = \frac{Frobenius(result_{standard} - result_{hardware})}{Frobenius(result_{standard})} \quad (4.3)$$

B. Mixed-Radix FFT Transformation Component

The supported FFT transformation scales in this study are 30×30 , 15×15 and 9×9 . These specific transformation scales are selected for two primary reasons: firstly, as presented in Fig. 1, the computational workload associated with small-size IFMs accounts for approximately 60%-80% of the entire network. Therefore, these selected particular transformation scales are highly suitable for effectively accelerating the small-size IFMs. Secondly, by splitting large-size IFMs into several tiles, the aforementioned three transformation scales also exhibit a significant acceleration effect on large-size IFMs. Based on these transformation scales mentioned above, the adaptation methods for different IFMs are provided in Table V. For the circumstance that IFM size greater than 30, they will be divided into multiple 30×30 tiles and then undergo a 30×30 FFT transformation. For IFMs whose size smaller than 30, they will be filled into the closest available transformation scale to minimize any introduction of invalid zero elements.

Next, Fig. 7 illustrates the overall structure of the FFT transformation module, which comprises five submodules: including zero padding, row data reordering, row FFT transformation, column data reordering and column FFT transformation. To perform 2D FFT transformation effectively, it is necessary to split it into two separate 1D transformations for rows and columns respectively; hence the reordering and FFT transformations should be conducted independently within the module.

$$\begin{aligned}
 Perf. &= \frac{1}{Single\ Comput.\ Time} = \frac{1}{\frac{Latency}{Comput.\ Num.}} = \frac{1}{\frac{Comput.\ Num. (Actual)}{DSP\ Num. * DSP\ Utilization * Fre.} * \frac{1}{Comput.\ Num. (Original)}} \\
 &= \frac{DSP\ Num. * DSP\ Utilization * Fre. * Comput.\ Num. (Original)}{Comput.\ Num. (Actual)} \quad (3.5)
 \end{aligned}$$

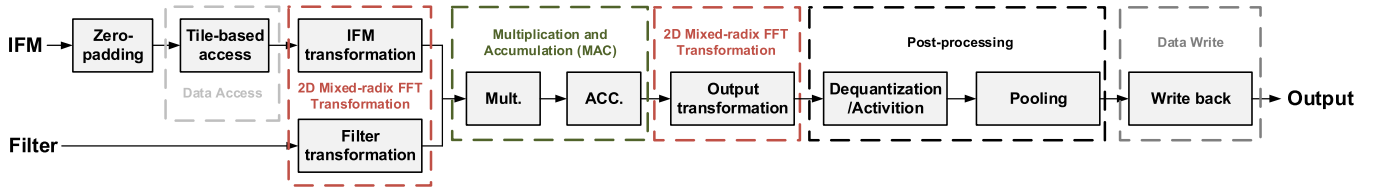


Fig. 5. Computation dataflow of this work.

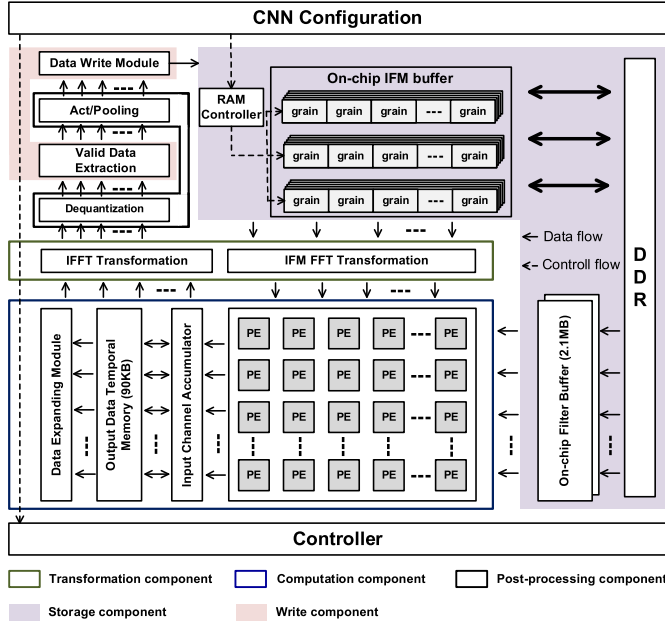


Fig. 6. Hardware architecture overview.

TABLE V

THE FFT TRANSFORMATION SCALE SUPPORTED BY OUR ACCELERATOR

Post-padding IFM	Transformation scale	Expanding/Splitting
$fea_size_padding > 30$	30×30	Splitting
$15 < fea_size_padding \leq 30$	30×30	Expanding
$9 < fea_size_padding \leq 15$	15×15	Expanding
$fea_size_padding > 9$	9×9	Expanding

After receiving the input tiles from the input buffer, the FFT transformation module initiates the zero-padding process through the zero-padding submodule. In the zero-padding module, the IFMs/input tiles will be padded to the specific scales (i.e., 30×30 , 15×15 or 9×9). Subsequently, the row reordering submodule arranges each row according to FFT requirements for post-padding IFMs/input tiles. Since the maximum transformation scale supported by this work is 30×30 , this module can perform 30 rows of reordering simultaneously. The following row FFT transformation module will perform parallel 1D FFT transformation for the reordered IFM/input tile rows. The function of the column reordering and FFT transformation module is identical to that of the row reordering/FFT transformation module, with the main distinction lying in their parallelism. With the aid of Hermitian symmetry, the parallelism of the column reordering module

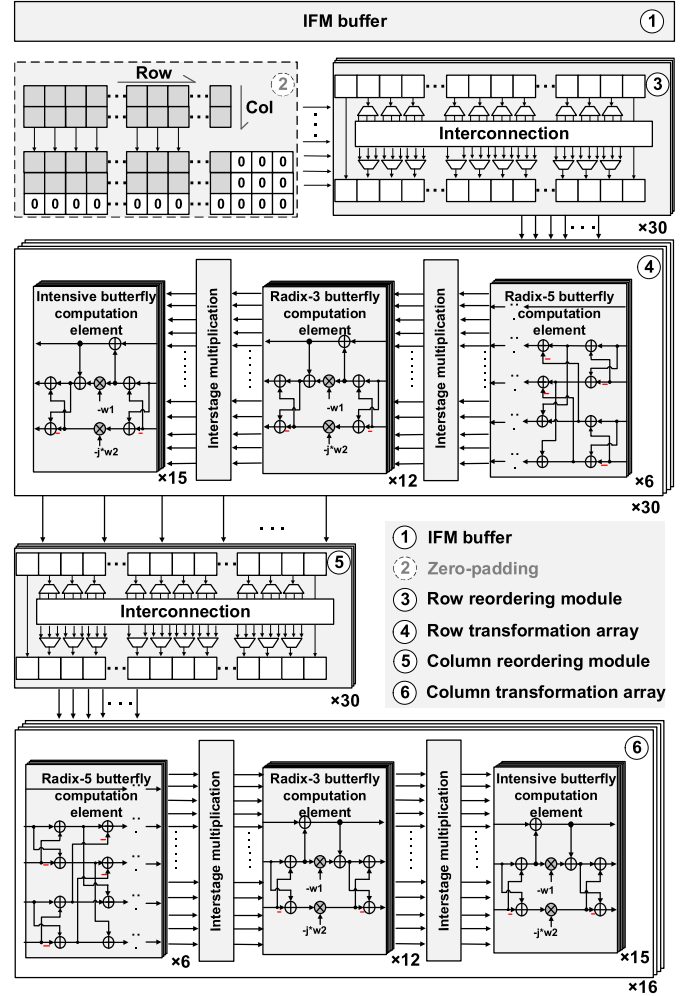


Fig. 7. FFT transformation module.

and FFT transformation module is 16 instead of 30, resulting in significant reduction in hardware overhead.

The row/column FFT transformation module implemented in this work is composed of multiple FFT transformation units with the same structure. The dataflow of the transformation unit is presented in Fig. 8, which includes two input ports (i.e., input port 0 and input port 1) and two output ports (i.e., output port 0 and output port 1). The transformation unit supports 30-point, 15-point and 9-point FFT. The black arrow represents the dataflow of 30-point FFT or 15-point FFT, while the gray arrow represents the dataflow of 9-point FFT. In the inter-stage rotation factor multiplication, the black letter represents the inter-stage rotation factor to be multiplied when making 15/30-point FFT, and the gray letter represents the inter-stage rotation factor to be multiplied when making 9-point FFT.

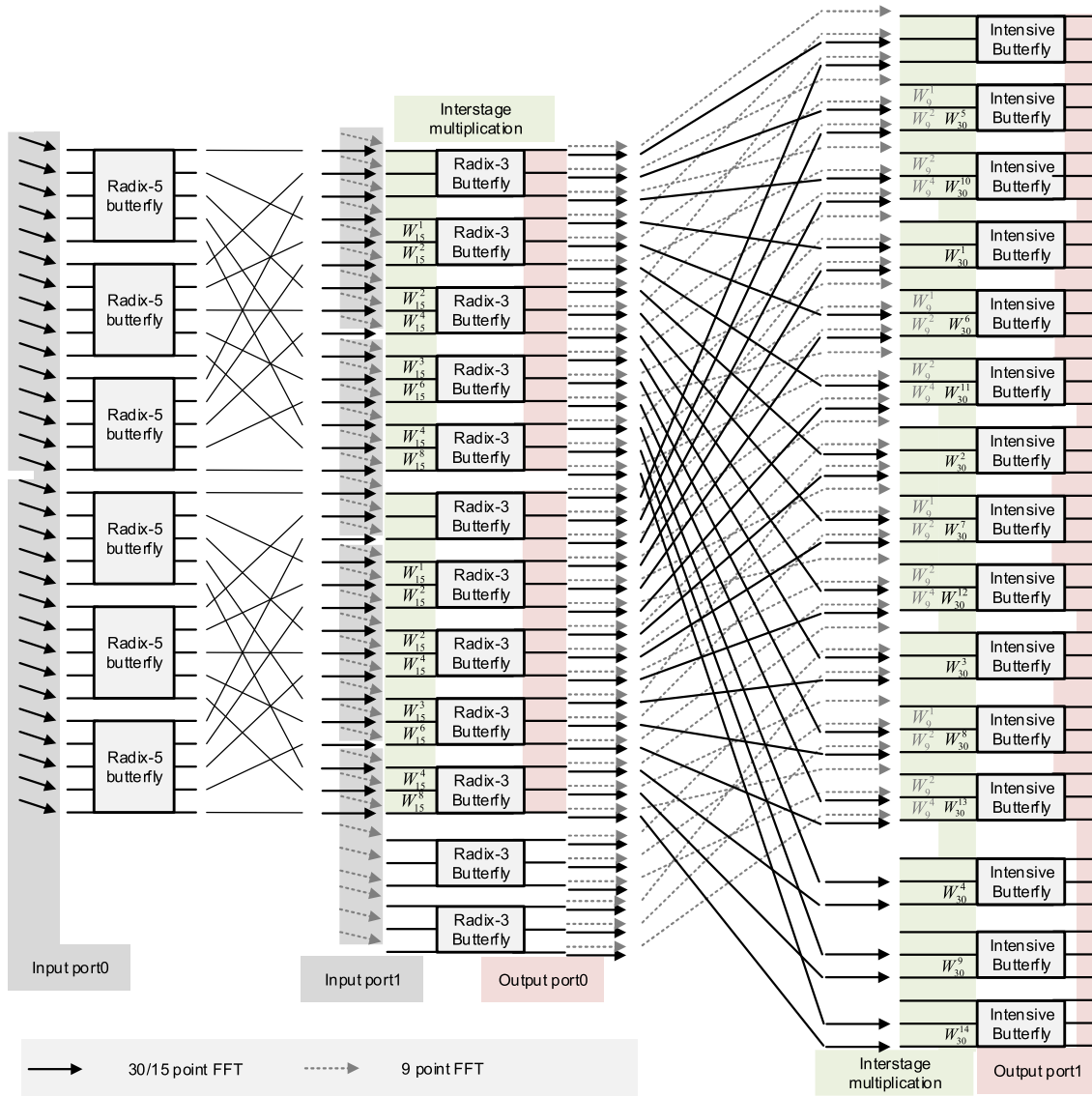


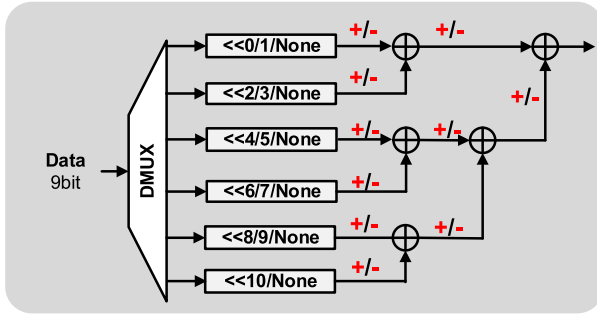
Fig. 8. Three-stage butterfly computation module.

The input data of 30-point FFT and 15-point FFT is received through the input port 0 of the transformation unit. When 30-point FFT is operated, a sequence of 30 points is submitted as the input data; while switching to 15-point FFT, two 15-point sequences should be sent to the input port 0. Following radix-5 butterfly computation element operations, the results are multiplied by the first interstage rotation factor and then transmitted to the radix-3 butterfly computation element. The results obtained by the radix-3 butterfly computation element would be output as the results of 15-point FFT or sent to take the second interstage multiplication to execute 30-point FFT. As for 15-point FFT, the results obtained by the radix-3 butterfly computation element are the transformation results of the above two 15-point sequences. Furthermore, in terms of 30-point FFT, the transformation results are obtained from the intensive butterfly computation element and output through port 1. The above intensive butterfly computation element is configured into the radix-2 butterfly computation element during the execution of 30-point FFT.

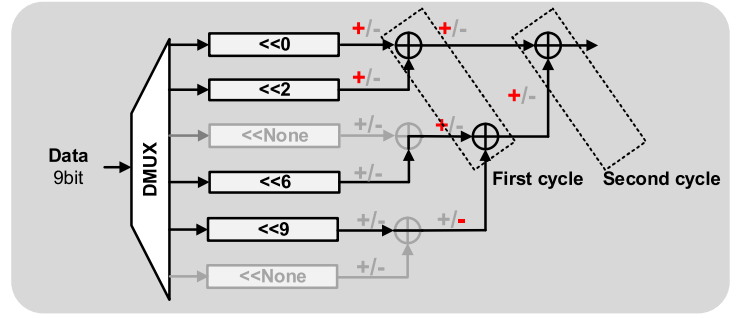
The input data of the 9-point FFT enters from the input port 1, and there are totally four sequences of 9 points operated in parallel. In this case, the first-level radix-5 butterfly operation is bypassed, and the input sequences are directly forward to the following radix-3 butterfly computation element. Then the results obtained by the above element are sent to be multiplied with the inter-stage rotation factor. Subsequently, the intensive butterfly computation element is configured into the radix-3 butterfly computation element. After the intensive butterfly operation, the final results are obtained as the transformation results of 9-point FFT.

C. Improvements on FFT Transformation by Booth Multiplier

In order to reduce the hardware overhead required by the FFT transformation module, Booth multiplier is employed for the implementation of the butterfly computation elements. In this study, Booth coding is utilized to obtain Table VI for handling in-stage rotation factors in radix-3 and radix-5 butterfly computation elements. Assuming the radix-4 Booth



(a) General structure for booth multiplier



(b) Booth multiplier for interstage multiplication

Fig. 9. The transformation improvement by using Booth multiplier.

TABLE VI
THE BINARY CODE AND BOOTH CODE OF INTRA-STAGE
ROTATION FACTORS

Intra-stage rotation factor	Binary code	Booth code
Radix-3: $\cos(2\pi/3)-1$	1010000000	-1 1 0 0 0 0
Radix-3: $-\sin(2\pi/3)$	11001000101	0 -2 1 0 1 1
Radix-5: $0.5(\cos(2\pi/5)+\cos((4\pi/5)))-1$	10110000000	-1 2 -2 0 0 0
Radix-5: $0.5(\cos(2\pi/5)-\cos((4\pi/5)))$	00100011110	0 1 0 2 0 -2
Radix-5: $0.5(\sin(4\pi/5)-\sin((2\pi/5)))$	11110100011	0 0 -1 -2 1 -1
Radix-5: $-0.5(\sin(2\pi/5)+\sin((4\pi/5)))$	11001110110	0 -2 2 -1 2 -2
Radix-5: $0.5(\sin(2\pi/5)-\sin((4\pi/5)))$	00001011101	0 0 1 2 -1 1

multiplier is used and denoting B as $b5b4b3b2b1b0$, each bit bi of B can take values from -2 to 2 . For $C \times B$, when $bi = 0$, this bit is directly skipped and no shift or addition operations are performed; when $bi \neq 0$, the i_{th} bit is shifted to C by $2i \times \log_2 |bi|$, and any non-zero results after shifting is added to obtain the final results. Taking the first interstage rotation factor $-w1$ of radix-3 as an example, the corresponding Booth encoded value is $\{-1 1 0 0 0 0 0\}$. For $C \times -\sin(2\pi/3)$, $C \ll 8$ is obtained from the second highest 1, $C \ll 10$ is obtained from the highest 1, therefore, the final result is $(C \ll 8) - (C \ll 10)$.

Fig. 9(a) is the general structure of Booth multiplier. It should be noted that the “None” value indicates that no operation is performed on the input, resulting in that the output is always 0. Consequently, this path can be disregarded. In order to further reduce calculations and save computing cycles, the shifters and related adders configured as “None” are directly eliminated. The multiplier of the second interstage rotation factor of the radix-3 butterfly computation element is shown in Fig. 9(b), thereby two adders are saved compared with Fig. 9(a). Subsequently, Fig. 10 illustrates the design of an intensive butterfly computation element where $w1$ and $w2$ correspond to those used in the radix-3 butterfly computation element and are also implemented using a Booth multiplier. This intensive butterfly computation element can be configured to reconstitute either a radix-3 butterfly computation element or a radix-2 butterfly computation element. When $S0$ is set to 0, the intensive butterfly computation element is configured as a radix-2 butterfly computation element, and data are input from in port 1 and in port 2 and output from out port 1 and out port 2. When $S0$ is configured as 1, the intensive radix-2

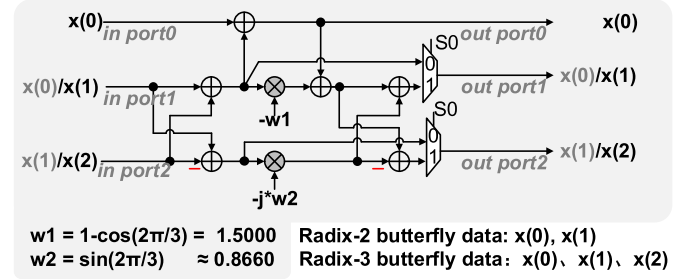


Fig. 10. Intensive butterfly computation element.

butterfly computation element switches to a radix-3 butterfly computation element, and data is input from in port 0, in port 1 and in port 2, and output from out port 0, out port 1 and out port 2.

D. The Structure of PE and PE Array

Regarding the dot-multiplications of the FFT-based CNN acceleration, each complex multiplication necessitates 4 real-number multiplications and 2 real-number additions. In terms of fixed-point computations, the implementation complexity of the multiplier is significantly higher than that of the adder. Therefore, in this study, an enhanced approach of complex multiplication is employed to substitute multipliers with adders, thereby achieving effective reduction in hardware overhead. Specifically, the conventional complex multiplication depicted in equation (4.4) can be replaced by the equation (4.5), where x is $a \times (c + d)$, y is $b \times (c - d)$, and z is $d \times (a + b)$. Therefore, at the cost of 3 additions, a reusable product z is obtained, thus reducing the number of real number multiplications from 4 to 3.

$$(a + bj)(c + dj) = (ac - bd) + (ad + bc)j \quad (4.4)$$

$$(a + bj)(c + dj) = (x - z) + (y + z)j \quad (4.5)$$

Fig. 11 shows the hardware structure of PE unit designed in our work. A single PE consists of 5 real adders, 3 real multipliers, multiple delay units and multiplexers. The adders and multipliers are utilized to implement the above multiplication reduction approach. The delay units can ensure dataflow synchronization by introducing a one cycle delay, and the multiplexers can change the data path by switching the intra-connections, so as to facilitate different functions by various configurations. The designed PE is capable of

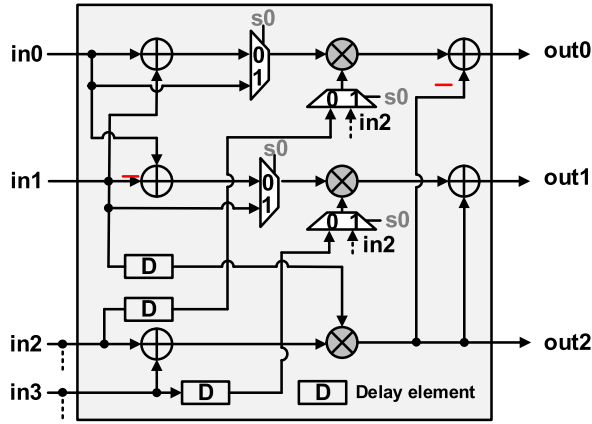


Fig. 11. The structure of PE.

performing either one complex dot-multiplication or two real dot-multiplications through configuration. When $s0 = 0$, the PE is configured to implement complex dot-multiplications. Assuming that after FFT transformation, the elements of IFMs are represented as $c + d \times i$ and the weights are represented as $a + b \times i$, the $in0$ and $in1$ ports of the PE are responsible for receiving the real part c and imaginary part d of the IFM elements respectively, while the $in2$ and $in3$ ports are responsible for receiving the real part a and imaginary part b of the weights. The implementation of the dot-multiplication function of the PE adopts the complex dot-multiplications. After performing the dot-multiplications, the real part of the dot-product is output through port $out0$, and the imaginary part of the dot product is output from port $out1$. When $s0$ transitions to 1, PE is configured to execute real-number dot-multiplications, which is applied to support the point-by-point convolution. In this case, the ports $in0$ and $in1$ of PE are responsible for receiving two elements of the IFMs, while port $in2$ simultaneously receive one weight. Subsequently, two IFM elements are dot-multiplied with one weight. At this time, $out0$ and $out1$ would output two single-products of point-by-point convolutions.

The PE array is composed of 32×16 PEs, and each PE can execute one complex dot-multiplication or two real dot-multiplications in one clock cycle. There are four available mapping modes for PE array: 30×16 , 15×8 , 7×5 , and $4 \times 4 + 5 \times 5$. Consequently, the PE array support one group of 30×16 multiplications, four groups of 15×8 multiplications, 12 groups of 7×5 multiplications, or 12 groups of $4 \times 4 + 5 \times 5$ multiplications in parallel. Table VII shows the compatibility of PE array across different convolution types. In this table, “general” represents regular convolutions whose filter size is 3×3 , 5×5 and etc., “point-wise” indicates point convolution whose filter size is 1×1 , and “rectangle” means the convolution whose filter size is $n \times m$ ($n \neq m$, e.g., 1×7 or 7×1). It is worth noting that with the help of Hermitian symmetry, only the first half of the columns of the transformed IFMs/input tiles need to be dot multiplied, while the dot-multiplication results of the second half of the columns can be directly obtained through the Hermitian symmetry of the first half of the columns. Therefore, taking 30×30 transformation scale as an example, only the first 16 columns need to be calculated, achieving significant multiplication savings.

TABLE VII

THE MAPPING OF DIFFERENT CONVOLUTION TYPES ON PE ARRAY

Mapped tile	Convolution type	Ideal mapped scale	PE parallelism
30×16	General	30×30	1
	Point-wise	30×30	1
	Rectangle	30 rows $\times$ 30 points	1
15×8	General convolution	15×15	4
	Point-wise	15×15	4
	Rectangle	15 rows $\times$ 15 points	4
7×5	Point-wise	7×7	12
	Rectangle	7 rows $\times$ 9 points	12
$4 \times 4 + 5 \times 5$	General convolution	9×9	12

Row		0	1	2	3	4	5	6	7	...	24	25	26	27	28	29	30	31	32	33
slice0	0	0	1	2	3	4	5	6	7	...	24	25	26	27	28	29	30	31	32	33
slice1	1	34	35	36	37	38	39	40	41	...	58	59	60	61	62	63	64	65	66	67
slice2	2	68	69	70	71	72	73	74	75	...	92	93	94	95	96	97	98	99	100	101
slice3	3	102	103	104	105	106	107	108	109	...	126	127	128	129	130	131	132	133	134	135
...																				
slice29	29	986	987	988	989	990	991	992	993	...	1010	1011	1012	1013	1014	1015	1016	1017	1018	1019
slice30	30	1020	1021	1022	1023	1024	1025	1026	1027	...	1044	1045	1046	1047	1048	1049	1050	1051	1052	1053
slice31	31	1054	1055	1056	1057	1058	1059	1060	1061	...	1078	1079	1080	1081	1082	1083	1084	1085	1086	1087
Column																				

Fig. 12. The mapping of IFM rows to different slices.

E. On-Chip IFM Buffer Structure

Compared to filter buffer, more IFM sizes and strides should be supported by the on-chip IFM buffer. Therefore, the structure and the read-write strategy of the on-chip IFM buffer are introduced in detail. The IFM buffer can be divided into two sub-buffers (buffer A and buffer B), which are used as the read buffer and the write buffer, respectively. During the operations of current layer, buffer A functions as the read buffer to store the IFMs of the current layer in the calculation process and provide them to the FFT transformation module and the following PE array; meanwhile, buffer B acts as the write buffer to store the output results. When transitioning to the subsequent layer, their roles are reversed with buffer A becoming the write buffer while buffer B serves as read buffer. The function exchange occurs whenever there is a transition between layers.

The structure of buffer A and buffer B is composed of 30 slices with the identical functionality and structure. When the IFM size is less than 30, each row of the IFMs will be stored in a corresponding slice on a one-to-one basis. When the IFM size exceeds 30, multiple rows of elements need to be accommodated within each slice. The mapping relationship from the IFM rows to the slice is depicted in Fig. 12. After storing the first 30 IFM rows (row 0 - row 29) to the slice0 to slice29, it is still necessary to store the 30th row in slice 0. Moreover, there are 30 grains in each slice and the width of each grain is 9-bit, so each grain can store a 9-bit data in an address. During the hardware implementation, it is observed that in the BRAM of XCVU440 has a depth of 1024 and a width of 36-bit. If a grain with a width of 9-bit is implemented

TABLE VIII
READ STRATEGY OF IFM BUFFER

Type	IFM size	Filter size	Stride	Output para.	Post-padding IFM size	t_size	fetch_step
1	$a > 30$	$3 \times 3 / 5 \times 5 / 7 \times 7$	1	1			
2	$26 < a \leq 30$	5×5	1	1	$a_p > 30$	30×30	$(30-k_size+1)$
3	$28 < a \leq 30$	3×3	1	1			
4	$11 < a \leq 15$, $15 < a \leq 26$	5×5	1	1	$15 < a_p \leq 30$	30×30	/
5	$13 < a \leq 15$, $15 < a \leq 28$	3×3	1	1			
6	$5 < a \leq 7$, $7 < a \leq 11$	5×5	1	4	$9 < a_p \leq 15$	15×15	/
7	$7 < a \leq 13$	3×3	1	4			
8	$a \leq 7$	3×3	1	12	$a_p \leq 9$	7×7	/

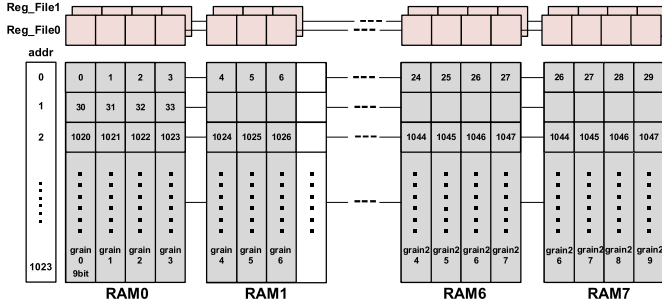


Fig. 13. The detailed structure of a slice in IFM buffer.

by a BRAM alone, a large amount of storage space will be wasted. Therefore, a BRAM is composed of 4 grains, and each slice requires 8 BRAMs in total. As shown in Fig. 13, RAM0 and RAM1 consists of 7 grains from grain0 to grain6, while RAM2 and RAM3 consists of 8 grains from grain7 to grain14, subsequently, RAM4, RAM5 and RAM6, RAM7 also consist of 7 and 8 grains from grain15 to grain21 and grain22 to grain29, respectively. To resolve the memory access conflict caused by simultaneously reading of the address 0 and address 1 in the BRAM as well as enable independent data reading for each individual grain, this paper designs two register files called Reg_File0 and Reg_File1, which make each grain be able to read data independently.

F. The Read/Write Strategy of IFM Buffer

After introducing the storage structure of IFM buffers, we will explain the read and write strategies of IFM in detail. According to the IFM size, filter size and stride, Table VIII classifies the reading strategy into 8 types. In the table, “a” represents the original IFM size, while “a_p” represents the size of the post-padding IFM. The fifth column in Table VIII is the output parallelism. Among these 8 types, post-padding IFM sizes are categorized into 4 cases: $a_p > 30$, $15 < a_p \leq 30$, $9 < a_p \leq 15$, and $a_p \leq 9$. These cases correspond to the 4 types of post-padding IFMs listed in Table V. As for general convolution, the parallelism of the input channel remains at 1, and the parallelism of the output channel can be 1, 4, and 12. The parallelism could be 4 and 12 because PE can operate 4 groups of 15×15 complex dot-multiplications and 12 groups of 9×9 complex dot-multiplications. For types 1 to 3, due to the large size of the post-padding IFMs, they need to be divided into multiple tiles. Moreover, for types 4 to 8, a bank can store all the data of an IFM, so all the data of the IFM are taken in parallel.

Subsequently, Table IX shows the write strategies for different IFM sizes. The write strategy is related to the size of the IFMs and the operation parallelism. For different IFM sizes, the operation parallelism has been determined, which is presented in Table IX. There are two write strategies designed in this work, including type 1 when the IFM size is greater than 30, and type 2 when the operation parallelism is 3. For write type 1, the output feature map is also divided into several output tiles, consequently, multiple clock cycles are required to write the complete results. For write type 2, a bank can store all the results of an IFM, thereby all of convolution results corresponding to an IFM can be written in just one cycle.

V. EXPERIMENTAL EVALUATION

In this section, first, regarding the hardware evaluation, a hardware report is provided to demonstrate the resource saving by using booth multiplier and to present the runtime frequency of our designed hardware. Moreover, a performance comparison with other current works, which exploits several convolution acceleration algorithms including Winograd, FFT and NTT, is presented to show the excellent performance and DSP efficiency of our designed accelerator.

A. Hardware Resource Utilization Evaluation

The improvement for FFT/IFFT transformation module using Booth multiplier is evaluated. The designed accelerator is synthesized and implemented on Xilinx XCVU440 FPGA, and the FFT and IFFT transformation modules are implemented by using general multipliers and Booth multipliers based on the VIVADO Integrated Development Environment (IDE). Table X presents a resource comparison for the FFT transformation module before and after employing Booth coding. It should be noted that the multipliers are implemented by LUTs rather than DSPs, consequently, the consumption of DSP slices is not listed. It can be observed that there is a reduction in hardware resource consumption for LUT, FF, and CARRY8 by more than 1/3 (35.89%, 37.98%, and 33.66% respectively) when using Booth multipliers. The reduction of hardware resources stems from the fact that the improved Booth multipliers effectively halves the number of “1” in the rotation factor, thereby enabling a 50% reduction in hardware resource utilization for multiplier implementation.

Furthermore, the improvement of IFFT transformation module is also discussed. By using Booth multipliers, the resource consumption can be effectively saved, which is shown in

TABLE IX
WRITE STRATEGY OF IFM BUFFER

Type	IFM size	Filter size	Stride	Post-padding IFM size	Write type	Tile para.	Output para.
1	$a > 30$	$3 \times 3 / 5 \times 5 / 7 \times 7$	1	$fea_out > 15$	Write type 1	1	1
2	$26 < a \leq 30$	5×5	1	$26 < fea_out \leq 30$			
3	$28 < a \leq 30$	3×3	1	$28 < fea_out \leq 30$			
4	$12 < a \leq 15,$ $15 < a \leq 26$	5×5	1	$12 < fea_out \leq 15,$ $15 < fea_out \leq 26$	Write type 2	1	1
5	$14 < a \leq 15,$ $15 < a \leq 28$	3×3	1	$14 < fea_out \leq 15,$ $15 < fea_out \leq 28$			1
6	$a = 12$	5×5	1	$fea_out = 12$			1
7	$a = 14$	3×3	1	$fea_out = 14$			3
8	$5 < a \leq 7,$ $7 < a \leq 11$	5×5	1	$5 < fea_out \leq 7,$ $7 < fea_out \leq 11$			4
9	$7 < a \leq 13$	3×3	1	$7 < fea_out \leq 13$			4
10	$a \leq 7$	3×3	1	$fea_out \leq 7$			12

TABLE X

THE RESOURCE COMPARISON FOR FFT TRANSFORMATION MODULE USING GENERAL MULTIPLIERS AND BOOTH MULTIPLIERS

Resource type	Multiplier	Booth multiplier	Saving ratio
LUT	449013	287860	35.89%
FF	479875	297621	37.98%
CARRY8	64647	42885	33.66%

TABLE XI

THE RESOURCE COMPARISON FOR IFFT TRANSFORMATION MODULE USING GENERAL MULTIPLIERS, BOOTH MULTIPLIERS AND BIT-WIDTH REDUCTION

Resource type	Multiplier	Booth multiplier	Booth multiplier and bit-width reduction	Saving ratio
LUT	1218737	748507	287860	23.62%
FF	1283418	755810	297621	23.19%
CARRY8	167374	96605	42885	25.62%

Table XI. However, the hardware overhead of the improved IFFT transformation is still significant. The basic structure of IFFT transformation is similar to FFT, the only difference is the bit-width of input data, which is 55-bit rather than 19-bit, thereby the resource consumption of IFFT transformation module is also 2.6 times higher than that of FFT transformation module. Consequently, after complex multiplication and channel accumulation, the results are firstly de-quantized to 9-bit, by decreasing the operated bit-width, the hardware resources of IFFT transformation module are reduced to the same as FFT transformation module, thus making the overall hardware overhead greatly reduced. Moreover, the bit-width reduction cannot be applied to the FFT transformation. During the process of dequantization module in our accelerator, the convolution results (i.e., the output results of the IFFT transformation module) are reduced to a lower bit-width representation as the input data for the next layer, which can avoid the bit-width expansion caused by multi-layer convolutions. Since the dequantization module has to discard many lower bits, it provides opportunity for us to discard some lower bits in advance at the IFFT stage, where these discarded bits are almost not affected after dequantization. In this way, the

bit-width of IFFT transformation module can be reduced so as to save hardware overhead. However, as for the FFT transformation, the results of FFT transformation don't need to be reduced to the lower bit-width, so the bit-width reduction method cannot be used to the FFT transformation. Fortunately, the data bit-width of the FFT stage is relatively lower, so that the hardware overhead of the FFT module is acceptable.

Finally, the hardware accelerator designed in this paper has a clock frequency of 238MHz. This high frequency achievement can be attributed to two factors: first, the multiplication and accumulation operations are divided to three and two clock cycles respectively, effectively reducing the logic latency; second, critical resources such as LUT and FF exhibit an optimal utilization rate of approximately 50%, thereby mitigating issues like implementation congestion and undesirable wire delay.

B. Comparison With Other Works

After reporting the hardware utilization, our proposed accelerator is also evaluated based on two classical CNNs (i.e., VGG16 and YOLOv3-tiny), additionally, the actual performance and DSP efficiency of our work are tested. Finally, the comparison results with current works that employ CNN acceleration methods (e.g., Winograd, FFT and NTT) are provided in Table XII. Actually, there are many differences among the above works and ours, such as benchmarks and accelerator types. As for the benchmarks, although multiple CNNs are applied in our works and others, we still try to take comparisons based on the same CNN model. In addition, for different accelerator types, we think that each accelerator has its own unique acceleration type which reflects the novelty and critical improvement of this study. Therefore, taking the comparison based on different accelerator types is to present the performance gain of the proposed mixed-radix FFT algorithm and designed architecture. Moreover, in terms of the applied FPGA platforms and clock speed, according to our experience, the choice of FPGA platform is a practical problem that needs comprehensive consideration. Therefore, the FPGA platforms of the compared works are various and it is hard to normalize. Additionally, it is found that the adoption of FPGA platform will mainly affect the operating frequency of hardware, Consequently, a new metric named frequency normalized performance (GOPS/MHz) is applied for further

TABLE XII
COMPARISONS WITH OTHER DESIGNS

	S. Kala [33] TVLSI 2019	C. Yang [8] TCAS-I 2019	Y. Liang [11] TCAD 2020	Y. Liang [34] TCAD 2022	Y. Xin [35] TC 2022	J. Jiang [36] TCAS-II 2022	S. Li [16] TVLSI 2022	Prasetyo [12] TCAS-I 2023	Ours
CNN model	AlexNet/VGG16	VGG16	VGG-16/YOLO	AlexNet	YOLOv3_tiny	VGG+BiLSTM	YOLOv3-tiny	VGG-16	VGG-16/YOLOv3-tiny
Accelerator type	Winograd-GEMM	Winograd	Radix-2 FFT	Radix-2 FFT	Winograd	Winograd	Winograd/Integer-frame reuse	NTT	Radix-2/3/5 FFT
Platform	FPGA VX690T	FPGA XCVU9P	FPGA ZC706/ZCU102	FPGA ZC706	FPGA XCKU115	FPGA Arria 10	FPGA ZCU104	FPGA Alveo U50	FPGA XCVU440
Clocks (MHz)	200	330	166	200	320	163	200	200	238
Precision	Fixed 16	Fixed 8	Fixed 16	Fixed 16	BFP 16	Fixed 16/8	Fixed 8	INT8	Fixed 9
DSP	1436	4096	900	900	1440	1161	146	4096	1536
LUT	468K	225K	350K	/	137K	/	123k	601.7K	1093K
FF	649K	/	/	/	675K	/	45K	810K	2160K
BRAM	1465	562	1090	/	768	1565	326	1084	1020
ECS	533K	/	/	/	357K	/	97K	758K	629K
Power (W)	17.2	35.0	23.6/9.4	6.1	13.6	26.6	3.30	26.0	44.1
Actual performance (GOPS)	433.63/407.23	5288	296.8/201.1	254.3	1115.3	1223.53	180.5/348.0	2859.5	2797/2109
Frequency normalized performance (GOPS/MHz)	2.168/2.036	16.024	1.788/1.211	1.272	3.485	7.506	0.903/1.740	14.298	11.752/8.861
DSP efficiency (GOPS/DSP)	0.302/0.284	1.291	0.330/0.223	0.290	0.775	1.054	1.236/2.384	0.698	1.82/1.37
ECS efficiency (GOPS/ECS)	0.814/0.764	/	/	/	3.124	/	1.861/3.588	3.772	4.447/3.353

evaluation and comparison. Finally, for different precision, it mainly impacts the hardware overhead of the implemented accelerator, which is already considered in the comparisons. Specifically, ECS (Equivalent FPGA CLB Slices), which is proposed in work [37] and represented as $(LUT / 8 + FF / 16) \times 97\% + DSP \times 102.4 + BRAM \times 196$, is applied and the further ECS efficiency (GOPS/ECS) is evaluated as a new metric for comparisons with other works.

Firstly, the actual performance of our implemented hardware is evaluated and compared with other works. In the case of VGG16, our designed hardware can achieve $2.29 \times - 9.42 \times$ speedup while comparing with previous works that utilize Winograd and radix-2 FFT [11], [33], [36]. Specifically, the conventional radix-2 FFT algorithm is applied by work [11], and our implemented hardware can realize $9.42 \times$ speedup, demonstrating the excellence of our proposed mixed-radix FFT algorithm and designed hardware accelerator. Similarly, in terms of YOLOv3-tiny, our proposed architecture can also realize up to $6.06 \times$ speedup, showing the excellent performance of the designed hardware accelerator. However, it is worth noting that the performance of works [8] and [12] is higher than that of our work, and this is mainly because of the larger computation array including 4096 DSPs they operate.

Moreover, DSP efficiency is also adopted to ensure a more equitable comparison. Consequently, while deploying VGG16, our proposed architecture can achieve $1.41 \times - 6.41 \times$ speedup on DSP efficiency compared to previous works [8], [11], [12], [33], and [36]. Furthermore, compared to work [35], our accelerator can also accomplish $1.77 \times$ speedup on DSP efficiency while deploying YOLOv3-tiny. It is noted that the DSP efficiency of work [16] is 1.74 times higher than that of our work, however, this is mainly because the similarity between the continuous frames in video is utilized in the above work, the DSP efficiency of this work is 1.236 GOPS/DSP when only Winograd algorithm is used, therefore, our designed hardware can also accomplish $1.11 \times$ speedup while compared with the only-Winograd version of work [16].

For fairer comparisons, we try to take all of the resources (i.e., LUT, FF, BRAM and DSP) into consideration. Specifically, ECS (Equivalent FPGA CLB Slices), which is proposed in work [37] and represented as $(LUT / 8 + FF / 16) \times 97\% + DSP \times 102.4 + BRAM \times 196$, is applied in this work. And the evaluation results are listed in Table XII. From this Table, it can be found that our work indeed has the strictest resource requirement, which is only lower than work [12]. However, with the aid of the advancement on actual performance, our work can still achieve competitive ECS efficiency (i.e., Actual performance/ECS), which is $1.18 \times - 5.82 \times$ better than works [12], [16], [33], [35]. We think this comparison result is fairer than the only performance comparison.

Through the above evaluation and comparisons, it is found that although designed hardware structure is somewhat complex, and the application of mixed-radix FFT and Hermitian symmetry always have more specific requirements compared to other algorithms (e.g., Winograd and NTT). However, we think that the significant gain on computation reduction of the mixed-radix FFT and the competitive performance of our proposed architecture should also not be ignored. Additionally, the area efficiency (i.e., ECS efficiency) of this work is also competitive. Consequently, in some scenarios where the inference latency requirement is severe but the requirement of algorithm complexity and hardware overhead are not severe, the designed mixed-radix FFT algorithm and hardware architecture still have a place.

VI. CONCLUSION

In this paper, a novel mixed-radix FFT algorithm is proposed to solve the mismatch between the fixed transformation scale provided by conventional radix-2 FFT algorithm and the various IFM sizes. Moreover, Hermitian symmetry is also utilized to further reduce the FFT transformation scale and required calculations. Furthermore, an efficient FFT-based CNN accelerator with a resource-efficient transformation component and multiplication-reduced PE array is designed. Our proposed accelerator is implemented on Xilinx XCVU440

platform with a running frequency of 238MHz, achieving 2109-2797 GOPS and 1.37-1.82 GOPS/DSP when VGG16 and YOLOv3-tiny are deployed. Compared with previous works that also employ convolution acceleration algorithms, the proposed accelerator can realize up to $9.42\times$ speedup on actual performance and $1.11\times - 6.41\times$ speedup on DSP efficiency, demonstrating the remarkable performance and excellent DSP efficiency of our work.

REFERENCES

- [1] K. He, X. Zhang, S. Ren, and J. Sun, "Delving deep into rectifiers: Surpassing human-level performance on ImageNet classification," in *Proc. IEEE Int. Conf. Comput. Vis. (ICCV)*, Dec. 2015, pp. 1026–1034.
- [2] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet classification with deep convolutional neural networks," in *Proc. 25th Int. Conf. Neural Inf. Process. Syst.*, New York, NY, USA, 2012, pp. 1097–1105.
- [3] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," 2014, *arXiv:1409.1556*.
- [4] J. Hong, B. Sapp, and J. Philbin, "Rules of the road: Predicting driving behavior with a convolutional model of semantic interactions," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2019, pp. 8446–8454.
- [5] M. Bojarski et al., "End to end learning for self-driving cars," 2016, *arXiv:1604.07316*.
- [6] J. R. R. Uijlings, K. E. A. van de Sande, T. Gevers, and A. W. M. Smeulders, "Selective search for object recognition," *Int. J. Comput. Vis.*, vol. 104, no. 2, pp. 154–171, Sep. 2013.
- [7] A. Lavin and S. Gray, "Fast algorithms for convolutional neural networks," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2016, pp. 4013–4021.
- [8] C. Yang, Y. Wang, X. Wang, and L. Geng, "WRA: A 2.2-to-6.3 TOPS highly unified dynamically reconfigurable accelerator using a novel Winograd decomposition algorithm for convolutional neural networks," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 66, no. 9, pp. 3480–3493, Sep. 2019.
- [9] C. Yang, Y. Wang, X. Wang, and L. Geng, "A stride-based convolution decomposition method to stretch CNN acceleration algorithms for efficient and flexible hardware implementation," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 67, no. 9, pp. 3007–3020, Sep. 2020.
- [10] T. Abtahi, C. Shea, A. Kulkarni, and T. Mohsenin, "Accelerating convolutional neural network with FFT on embedded hardware," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 26, no. 9, pp. 1737–1749, Sep. 2018.
- [11] Y. Liang, L. Lu, Q. Xiao, and S. Yan, "Evaluating fast algorithms for convolutional neural networks on FPGAs," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 39, no. 4, pp. 857–870, Apr. 2020.
- [12] S. Hong, Y. F. Arthanto, and J.-Y. Kim, "Accelerating deep convolutional neural networks using number theoretic transform," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 70, no. 1, pp. 315–326, Jan. 2023.
- [13] J. Yezep and S.-B. Ko, "Stride 2 1-D, 2-D, and 3-D Winograd for convolutional neural networks," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 28, no. 4, pp. 853–863, Apr. 2020.
- [14] Y. Meng, C. Yang, S. Xiang, J. Wang, K. Mei, and L. Geng, "An efficient CNN accelerator achieving high PE utilization using a dense-/sparse-aware redundancy reduction method and data-index decoupling workflow," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 31, no. 10, pp. 1537–1550, Oct. 2023.
- [15] S. Winograd, *Arithmetic Complexity of Computations*, vol. 33. Philadelphia, PA, USA: SIAM, 1980.
- [16] S. Li, Q. Wang, J. Jiang, W. Sheng, N. Jing, and Z. Mao, "An efficient CNN accelerator using inter-frame data reuse of videos on FPGAs," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 30, no. 11, pp. 1587–1600, Nov. 2022.
- [17] X. Liu et al., "Efficient sparse-Winograd convolutional neural networks," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2018, pp. 1–4.
- [18] S. Li, J. Park, and P. T. P. Tang, "Enabling sparse Winograd convolution by native pruning," 2017, *arXiv:1702.08597*.
- [19] D. Wu, X. Fan, W. Cao, and L. Wang, "SWM: A high-performance sparse-Winograd matrix multiplication CNN accelerator," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 29, no. 5, pp. 936–949, May 2021.
- [20] X. Wang, C. Wang, J. Cao, L. Gong, and X. Zhou, "WinoNN: Optimizing FPGA-based convolutional neural network accelerators using sparse Winograd algorithm," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 39, no. 11, pp. 4290–4302, Nov. 2020.
- [21] L. Lu and Y. Liang, "SpWA: An efficient sparse Winograd convolutional neural networks accelerator on FPGAs," in *Proc. 55th ACM/ESDA/IEEE Design Autom. Conf. (DAC)*, San Francisco, CA, USA, Jun. 2018, pp. 1–6.
- [22] H. Wang, W. Liu, T. Xu, J. Lin, and Z. Wang, "A low-latency sparse-Winograd accelerator for convolutional neural networks," in *Proc. IEEE Int. Conf. Acoust., Speech Signal Process. (ICASSP)*, May 2019, pp. 1448–1452.
- [23] C. Yu and M.-H. Yen, "Area-efficient 128- to 2048/1536-point pipeline FFT processor for LTE and mobile Wimax systems," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 23, no. 9, pp. 1793–1800, Sep. 2015.
- [24] X.-Y. Shih, H.-R. Chou, and Y.-Q. Liu, "VLSI design and implementation of reconfigurable 46-mode combined-radix-based FFT hardware architecture for 3GPP-LTE applications," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 65, no. 1, pp. 118–129, Jan. 2018.
- [25] Y.-C. Lee, T.-S. Chi, and C.-H. Yang, "A 2.17-mW acoustic DSP processor with CNN-FFT accelerators for intelligent hearing assistive devices," *IEEE J. Solid-State Circuits*, vol. 55, no. 8, pp. 2247–2258, Aug. 2020.
- [26] T. Abtahi, A. Kulkarni, and T. Mohsenin, "Accelerating convolutional neural network with FFT on tiny cores," in *Proc. IEEE Int. Symp. Circuits Syst. (ISCAS)*, Baltimore, MD, USA, May 2017, pp. 1–4.
- [27] C. Fang, L. He, H. Wang, J. Wei, and Z. Wang, "Accelerating 3D convolutional neural networks using 3D fast Fourier transform," in *Proc. IEEE Int. Symp. Circuits Syst. (ISCAS)*, May 2021, pp. 1–5.
- [28] L. He, X. Xie, J. Lin, and Z. Wang, "Efficient FPGA design for convolutions in CNN based on FFT-pruning," in *Proc. IEEE Asia-Pacific Conf. Circuits Syst. (APCCAS)*, Ha Long, Vietnam, Dec. 2020, pp. 27–30.
- [29] C. Yang, J. Wu, S. Xiang, L. Liang, and L. Geng, "A high-throughput and flexible architecture based on a reconfigurable mixed-radix FFT with twiddle factor compression and conflict-free access," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 31, no. 10, pp. 1472–1485, Oct. 2023.
- [30] S. Liu and D. Liu, "A high-flexible low-latency memory-based FFT processor for 4G, WLAN, and future 5G," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 27, no. 3, pp. 511–523, Mar. 2019.
- [31] X.-Y. Shih, H.-R. Chou, and Y.-Q. Liu, "Design and implementation of flexible and reconfigurable SDF-based FFT chip architecture with changeable-Radix processing elements," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 65, no. 11, pp. 3942–3955, Nov. 2018.
- [32] K.-F. Xia, B. Wu, T. Xiong, and T.-C. Ye, "A memory-based FFT processor design with generalized efficient conflict-free address schemes," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 25, no. 6, pp. 1919–1929, Jun. 2017.
- [33] S. Kala, B. R. Jose, J. Mathew, and S. Nalesh, "High-performance CNN accelerator on FPGA using unified Winograd-GEMM architecture," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 27, no. 12, pp. 2816–2828, Dec. 2019.
- [34] Y. Liang, Q. Xiao, L. Lu, and J. Xie, "FCNNLib: A flexible convolution algorithm library for deep learning on FPGAs," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 41, no. 8, pp. 2546–2559, Aug. 2022.
- [35] Y. Xin, D. Chen, C. Zeng, W. Zhang, Y. Wang, and R. C. Cheung, "High throughput hardware/software heterogeneous system for RRPn-based scene text detection," *IEEE Trans. Comput.*, vol. 71, no. 7, pp. 1507–1521, Jul. 2022.
- [36] J. Jiang, M. Jiang, J. Zhang, and F. Dong, "A CPU-FPGA heterogeneous acceleration system for scene text detection network," *IEEE Trans. Circuits Syst. II, Exp. Briefs*, vol. 69, no. 6, pp. 2947–2951, Jun. 2022.
- [37] W. Liu, S. Fan, A. Khalid, C. Rafferty, and M. O'Neill, "Optimized schoolbook polynomial multiplication for compact lattice-based cryptography on FPGA," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 27, no. 10, pp. 2459–2463, Oct. 2019.
- [38] Z. Song et al., "DRQ: Dynamic region-based quantization for deep neural network acceleration," in *Proc. ACM/IEEE 47th Annu. Int. Symp. Comput. Archit. (ISCA)*, Valencia, Spain, May 2020, pp. 1010–1021.
- [39] Y.-H. Chen, T.-J. Yang, J. Emer, and V. Sze, "Eyeriss v2: A flexible accelerator for emerging deep neural networks on mobile devices," *IEEE J. Emerg. Sel. Topics Circuits Syst.*, vol. 9, no. 2, pp. 292–308, Jun. 2019.